

Report for assignment 3

Project

Name: Android

URL: <https://github.com/CatimaLoyalty/Android/tree/main>

Description: Catima is an Android App that stores the store loyalty and membership cards on your device, instead of having to carry them around.

1. Onboarding experience

(a) Did you have to install a lot of additional tools to build the software?

Yes, we have to install the Android SDK in order to successfully build this project.

(b) Were those tools well documented?

No, the document doesn't mention installing any additional tools. It only mentions running the `./build.sh` script to build the project.

(c) Were other components installed automatically by the build script?

No, there are no additional components being installed by the build script.

(d) Did the build conclude automatically without errors?

The first time we ran the build script, it failed because neither of us had the Android SDK installed. Additionally, the script requires specific paths to locate the Java version being used and the Android SDK directory. The build script attempts to resolve this by setting the path variables to the locations where these would typically be located on a Linux machine. However, since neither of us uses Linux, this did not work. It also required Java 21, which had to be installed for those of us who didn't already have it. We had to adjust these settings before we could successfully execute the script, and after doing so, the build script ran without any issues.

(e) How well do examples and tests run on your system(s)?

We can not find any examples.

When the environment is correctly configured the tests run successfully, but the initial `./gradlew testReleaseUnitTest` instruction from the "Contributing.md" document does not work. Instead the project has different test flavors, one of them being the free open source version and the other being the google play version. The free open source version runs successfully via `./gradlew testFossReleaseUnitTest`.

2. Complexity

2.1. What are your results for five complex functions?

* Did all methods (tools vs. manual count) get the same result?

- F1: Yes, both manual and the tool got the same complexity.
Tool (lizard): 16
Manual: 16
- F2: Yes, both manual and the lizard tool got the same complexity result.
Tool (lizard) : 22
Manual: 22
- F3: Yes, both manual and the tool reached the same complexity, 19.
- F4:
 - lizard: 19, manual: $pi = 5(\text{if/for statements}) + 6(\text{cases in first switch}) + 7(\text{cases in second switch})$, $s = 1$, $18 - 1 + 2 = 19$
- F5: Yes, same results with manual vs tool. 19 in both cases.

*** Are the results clear?**

- F1: At first, I missed a ternary operator inside a function call, but when reading through the code more carefully I noticed it and the manually calculated CC got the same result as the tool. The result is straightforward, as the function mainly consists of conditional branching.
- F2: At first, I was unsure about the try-catch, but it turns out it's similar to the if-else statement.
- F3: Using $CC = P + 1$, where P represents decision points, made the results for this function clear.
- F4:
 - I was a bit unsure about switch statements, but it seems they work similarly to more standard statements, being based on the nr of decision points. I originally thought that exceptions count as a return point, in which case the manual result would be 17. When I realised they did not the results seemed clear
- F5: I was a bit off in my first calculation because there were some ternary operators that I did not notice. I also didn't realise that a composite boolean expression results in multiple branches. After taking this into consideration, I got the same results as the tool.

2.2. Are the functions just complex, or also long?

- F1: The function has 116 lines of code, so both complex and long.
- F2: This function has 108 lines of codes, which is considered long.
- F3: This function has 104 lines of code, so both complex and long.
- F4: The function has 112 lines of code, so both complex and long.
- F5: The function has 91 lines of code, so quite long and complex.

2.3. What is the purpose of the functions?

- F1: The function sets up the UI for viewing a card when the activity is resumed. It sets the brightness, disables the lock screen, adds title, fixes on-click functionality, sets the colors, checks barcode validity, and sets front and back images.

- F2: The purpose of this function, onClick under chooseImage class, is to check whether the corresponding card is clickable according to the view id that was given and lastly generate corresponding dialog on the screen.
- F3: This function imports a single loyalty card into the database using the given session. It reads a single CSVRecord (comma separated value, single line of text) that represents an entry in a dataset and translates it to a LoyaltyCard object that the application can store or use in some way.
 - fetch the id from the CSV record
 - check store, expiration dates
 - set currency, balance, barcode and card metadata
- F4:
 - The function takes a raw JSONArray and transforms it into usable data for creating the cards. At the end of the function the cards are added to a list and returned via an ImportedData object. There are 5 main components, 1) extracting data such as store name and expiry date, 2) transforming a "codeType" string into a CatimaBarcode object 3) mapping a "color" string to a cards header color 4) taking the parsed values to create a LoyaltyCard object 5) adding LoyaltyCards to a list and finally returning as an ImportedData object
- F5: This method is invoked when the edit activity resumes and is responsible for restoring and synchronising the UI with the persisted state of the loyalty card. It updates all relevant visual components, including fields, images, barcode, groups, colors, and event listeners.

2.4. Are exceptions taken into account in the given measurements?

- F1: No, no exceptions are thrown in the method.
- F2: No, the exceptions are not taken in the measurements, only the branches are taken.
- F3: Yes, it looks like the lizard tool is counting catch-blocks (7) as decision points in this function. The Jacoco tool arrives at CCN = 12, which is the same CCN excluding all catch-blocks.
- F4: No
- F5: The method doesn't throw any exceptions.

2.5. Is the documentation clear w.r.t. all the possible outcomes?

- F1: No, the only documentation for this function are a few comments inside the function.
- F2: No, there's no documentation for this specific function and class.
- F3: No, there is no documentation for this function. A few comments explain very little about imports in respect to older versions.
- F4:
 - The only documentation for this function seems to be a [link](#) to an issue where the admin asks a person who did a similar project what export format they used

- F5: No, there is very limited documentation for this function. There are 4 comments in total. A Javadoc would have been nice to quickly get an idea of what the method does, given its length and complexity. That said, with previous knowledge of Android development, understanding the method is likely easier.

3. Refactoring

Plan for refactoring complex code:

Estimated impact of refactoring (lower CC, but other drawbacks?).

- F1: Currently, the function does a lot of different configurations. To reduce CC and make the function more readable, several of these configurations could be moved out to helper functions. More specifically, the following can be moved to helper functions:
 - The window configuration at the beginning of the function,
 - Setting the background colors, and
 - Validating the barcode + setting front and back images.
- F2: There are a lot of if-else statements for setting up the parameters based on the input id. Hence, it would be nice to create another helper function which takes in id and returns the configured parameters so the function itself can be clean, readable and also reduce CC.
- F3: Since this function is large, 104 LOC, the split method refactoring could be used. The plan would be to move the logic related to the metadata of the loyaltycard: headerColor, starStatus, archiveStatus, lastUsed. Lines 470-501 could be moved to a private helper called extractMetadata returning an object holding those values.
- F4:
 - Instead of using switch, we could extract the barcode logic into a helper function by defining an interface with different states. This would make it so the decisions are handled by an internal hash table instead of multiple if/else statements, reducing the code complexity significantly. [source](#)
 - This makes it easier to add new states without increasing complexity. However, for this case it is probably fairly rare that new states would be added/changed (= new bar code type invented, more colors), so a simpler approach might be more appropriate
- F5: The method currently rehydrates the UI with the persisted state of the loyalty card when the edit activity resumes. In general, most of this logic is related since it all updates the UI in some way, however, it currently mixes multiple concerns, which could be moved out to helper methods to reduce CC. This would include changing the following:
 - Extract binding logic where a value is read from viewModel.getLoyaltyCard().someValue and then set in the UI
 - Extract group-chip construction into a helper to isolate nested logic.

- Isolate header colour selection and thumbnail styling into dedicated helpers.

Carried out refactoring (optional, P+):

```
```bash
git diff main..refactoring
```
```

4. Coverage

Tools

Document your experience in using a "new"/different coverage tool.

- android studios built-in coverage tool: 103/103 tests failed.. probably some jdk version related error
- Jacoco: Some configuration needed to be set in *build.gradle.kts*, as well as saving the report. After that, Jacoco will generate an UI while running the tests for the test coverage results.

4.1 How well was the tool documented?

Jacoco is well-document for installation and configuration, however, it would be great to have documentation for the meaning of the result table so that a user can understand better what each column means and fortify the tests better.

4.2 Was it possible/easy/difficult to integrate it with your build environment?

- We needed to do manual changes to the *build.gradle.kts* to activate the Jacoco plugin and define the exact paths for the compiled class files (fossDebug).

Your own coverage tool

4.3 Show a patch (or link to a branch) that shows the instrumented code to gather coverage measurements. The patch is probably too long to be copied here, so please add the git command that is used to obtain the patch instead:

```
```bash
git diff main..development
```
```

4.4 What kinds of constructs does your tool support, and how accurate is its output?

- The accuracy of our tool depends on where the flags have been manually added in the functions. These are the decision points that we have taken into consideration:
 - if
 - else
 - catch
 - for-loops
 - while-loops

- ternary-operators
- switch/case

Evaluation

4.5.1. How detailed is your coverage measurement?

- It covers a similar level of detail as jacoco, as we are manually setting flags for every branch taken.

4.5.2. What are the limitations of your own tool?

- At the moment flags need to be set manually in each function that we want to check coverage for. Furthermore, in the case of loops, they might need to be wrapped in an if statement to check whether they execute once or not at all.

4.5.3. Are the results of your tool consistent with existing coverage tools?

- We found that our results are largely consistent with established tools such as JaCoCo. The main difference lies in how boolean expressions using short-circuit operators (&&, ||) are handled. JaCoCo treats the short-circuit evaluation paths of each operand as separate branches. For example, in if (cond1 && cond2), JaCoCo distinguishes multiple control-flow paths generated by evaluating cond1 and then evaluating cond2, effectively resulting in more reported branches. In contrast, our tool considers only the overall outcome of the compound predicate (true or false) as the relevant branches.

5. Coverage improvement

5.1 Show the comments that describe the requirements for the coverage.

Test cases added:

```
```bash
git diff main..after-tests
```
```

- F1: Tests in LoyaltyCardViewActivityTest.java
 - barcodeNotSupportedShowsToast
 - loyaltyCardNullShowsToast
 - frontImageBitmapNotNull
 - backImageBitmapNotNull
- F2: Tests in LoyaltyCardEditActivityTest.java
 - onClick_FrontImageHolder_ShowDialogCorrectly
 - onClick_backImageHolder_ShowDialogCorrectly
 - onClick_thumbnail_ShowDialogCorrectly
 - onClick_InvalidImageId_ThrowsException

- F3: Tests in ImportExportTest.java. Improve coverage from 86% to 100%.
 - missingStore
 - importInvalidBalanceFormat
 - missingCardId
 - importWithArchivedStatus
- F4: Tests in ImportExportTest.java
 - voucherVaultBarcodeFormatsMapCorrectly
 - voucherVaultMilliunitBalanceConvertsToBigDecimal
- F5: Tests in LoyaltyCardEditActivityTest. Improves coverage from 58% to 88%.
 - onResume_noChipsSameGroupId_addsCheckedChip
 - onResume_noChipsEmptyLoyaltyCardGroups_AddsUncheckedChip
 - onResume_balanceIsNull_setsBalanceToZero
 - onResume_whiteHeaderColor_setsDarkForeground

6. Self-assessment: Way of working

Current state according to the Essence standard:

- For this assignment we had to familiarize ourselves with new tools in a short time span, so the adaptation was less natural than the previous assignment. We therefore feel that we have "gone back" some steps, landing solidly in the "In place" stage but not quite reaching Working well.

6.1 Was the self-assessment unanimous?

- Yes, we pretty much agree on what is working well and what needs to be adjusted or improved upon.

6.2 Any doubts about certain items?

- We are unsure about if the tools naturally support us. We are very familiar with GitHub at this point, but for other tools and practices used during the assignment time constraints made it difficult to properly evaluate and refine our ways.

6.3 How have you improved so far?

- Overall, we have improved our communication and become more familiar with Github and proper practice such as conventional commits and templates as well as integrating different tools and libraries to support our work depending on what the assignment requires.

6.4 Where is potential for improvement?

- You can always improve communication in the team. As mentioned, we feel that time constraints have limited us from experimenting more with tools and evaluating practices.

7. Overall experience

7.1 What are your main take-aways from this project?

- Metrics provide a valuable guide for a developer, giving an overview of progress and the overall state of the program. Code complexity can give a sense of what areas might be at risk of bugs or harder to maintain, while

branch coverage can show what parts of the program are less tested. But we also need to create and analyze tests consciously; it would be easy to create tests covering all branches with simple assertions, but if they do not test specific features in a qualitative way it might just lead to a false sense of security.

7.2 What did you learn?

- By creating our own coverage measurement tools and manually flagging decision points in the 5 functions, we got a greater sense of how data actually travels through the system and how this is connected to the creation of tests. By analyzing the code complexity, the impact of refactoring a function became more clear.

8. Statement of Contribution

Emma Tisell (etisell@kth.se):

note: aiming for P+ grade

- Identified branches, wrote tests, and did refactoring for onResume in LoyaltyCardViewActivity (Func 1)
 - Improve coverage from 65% to 80%
 - Reduce CC from 16 to 4
 - The specific tests can be found in section 5.1.
- Moved tests from branch 'development' to 'after-tests'
- Corresponding documentation (JavaDoc)
- Code reviews + merging pull requests

Chih-Yun Liu (cylui4@kth.se):

note: aiming for P+

- Implemented the CoverageTool ShutdownHook
- Identified branches, wrote tests, and did refactoring for onClick in ChooseCardImage (Func 2)
 - Wrote 4 tests which improves coverage from 0% to 45%
 - Carrying out the refactoring which reduces CCN from 22 to 13 (40%)
 - The specific tests can be found in section 5.1.
- Corresponding documentation (JavaDoc)
- Code reviews + merging pull requests

Jacob Friedrich (jafr@kth.se):

note: aiming for P+ grade

- Identified branches, wrote tests, and did refactoring for importLoyaltyCard in ImportCatima (Func 3)
 - The specific tests can be found in section 5.1.
 - Improve coverage from 86% to 100%
 - Reduce CC from 19 to 11
- Corresponding documentation
- Code review + merging pull requests

Filippa Ciuk Olsson (fico@kth.se):

note: aiming for P grade

- Identified branches and wrote 2 tests for importJSON in VoucherVaultImporter (Function 4)
 - Improved branch coverage from 69% to 89%
 - The specific tests can be found in section 5.1.
- Corresponding documentation (JavaDoc)
- Code reviews + merging pull requests

Rasmus Sjöberg (rassjo@kth.se):

note: aiming for P+ grade

- Implemented the CoverageTool
- Identified branches, wrote tests, and did refactoring for onResume in LoyaltyCardEditActivity (Func 5).
 - Improve coverage from 58% to 88%
 - Reduced CCN from 19 to 4
 - The specific tests can be found in section 5.1.
- Code reviews + merging pull requests